ML-Based Predictive Cloud Resource Management

With SLA-Aware Auto-Scaling, Cost Optimization, and Adaptive Workload Handling

Student	Roll number
Parth Dhande	16010123232
Purab Madhan	16010123242
Pushkar Rai	16010123243

Abstract

We implemented and demonstrated a workload predictor and resource controller for a real CPU-bound HTTP service on a local Kubernetes cluster. The controller forecasts near-future request demand, adjusts application replicas, and combines prediction with observed-demand and latency safeguards. Controlled synthetic traffic is replayed as actual HTTP requests; latency, errors, worker CPU and replica changes are measured.

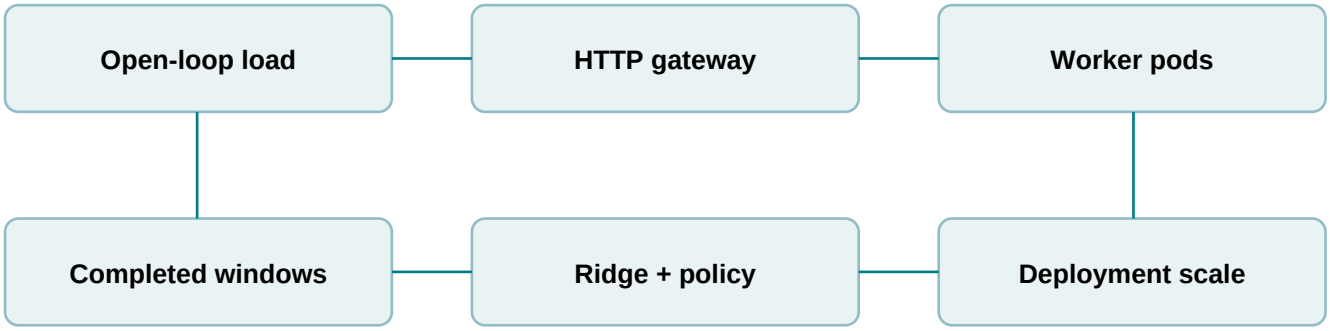
Evaluation compares fixed maximum capacity, custom reactive scaling and guarded predictive scaling on identical arrival schedules, with two independent seeds per workload. We report both performance and resource overhead, rather than assuming prediction is universally superior. Infrastructure cost and energy are transparent analytical proxies; they are not measured cloud bills or physical energy savings.

Scope and implementation status

The comparison suite contains **12 completed Kubernetes runs**. A trained Ridge model, raw request records, decision logs, resource timelines and reproducible commands accompany the code. The implementation scales application pods on one laptop, not physical nodes or an AWS Auto Scaling group.

Submission repository: github.com/parth1056/accml
Default branch: parth/predictive-scaler. Code baseline: 3c87760. Prepared from execution on 24 October 2026.

Architecture and execution platform



The host runs the traffic generator and controller. Requests reach one gateway pod through a local port-forward. The gateway opens a new connection to the worker Service for each request, allowing Kubernetes to distribute traffic across Ready endpoints. Scaling commands target only the dedicated acc-lab context/namespace. A local API proxy supplies pod state and sampled worker CPU metrics.

Component	Implementation
Platform	Docker Desktop Linux engine; single-node Minikube, Kubernetes v1.34.0; preloaded application image.
CPU task	PBKDF2-HMAC-SHA256, fixed 60,000 iterations/request; one active task per worker with a bounded asynchronous queue.
Resource envelope	Each worker requests/limits 250m CPU; 32 MiB memory request and 96 MiB limit. Primary comparisons allow 1-4 worker replicas.
Readiness/draining	Configured five-second readiness delay, identical for all policies; pre-stop delay and graceful drain reduce interrupted in-flight work.
Application interface	/work, /health, /ready and /metrics. Gateway and experiment viewer expose only local host connections.

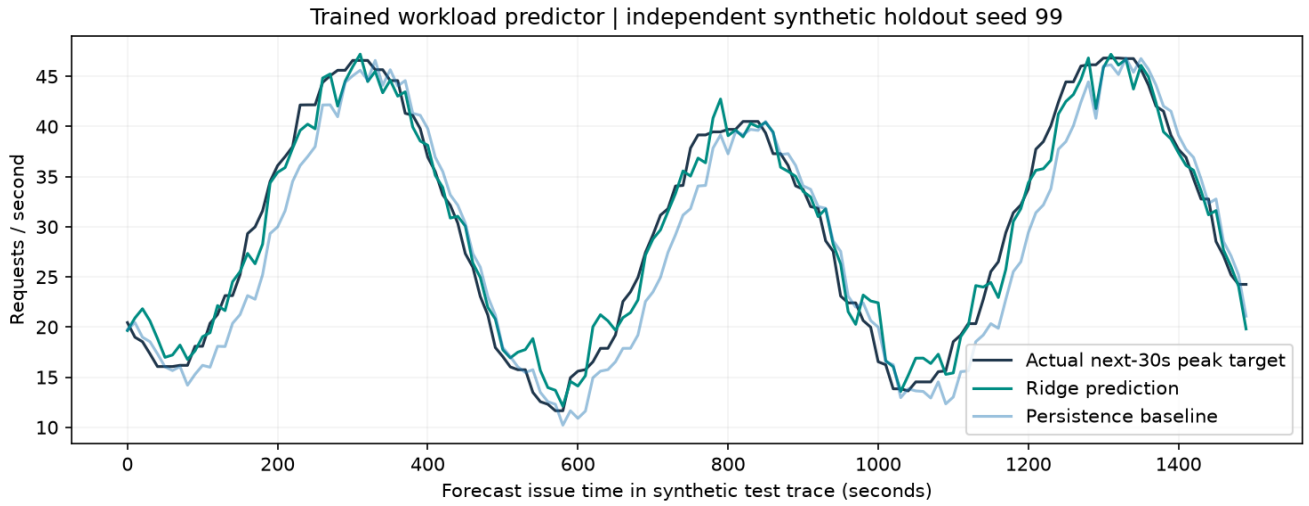
All replicas share the same physical host. Pod scaling redistributes a fixed host resource pool; reductions in allocated pods do not power off the laptop. The source includes a real-process backend for development, but the main comparison results in this report use Kubernetes.

Input data, features and ML training

Incoming demand is offered requests/second, counted from actual generator send timestamps in completed ten-second windows. Completed throughput is not used as demand because overload could suppress completions. The generator follows a seeded open-loop arrival schedule and records send lag, status and end-to-end latency for every request.

Item	Definition
Features (10)	Six completed RPS windows; trailing mean, standard deviation and maximum; short trend (latest minus oldest)/5.
Target	Maximum offered RPS across the next three ten-second windows: a 30-second provisioning horizon.
Algorithm	Standardized Ridge regression with an unpenalized intercept; negative predictions are clipped to zero.
Data separation	8544 training rows from seeds 0-11; 4272 validation rows from seeds 50-55; independent synthetic holdout seed 99. No labels cross sequence boundaries.
Selection	Validation selects alpha=10 from 1, 10 and 100. A validation positive-residual 90th percentile supplies empirical headroom, not guaranteed interval coverage.
Inference	The saved JSON coefficients are reused for matching capacity/horizon settings. This is offline training followed by online prediction, not continuous retraining.

On the separate synthetic holdout, Ridge MAE was **1.67 RPS** and RMSE **2.07 RPS**, versus persistence MAE **2.61 RPS**. These are model-only synthetic results; real HTTP replay results are evaluated separately.

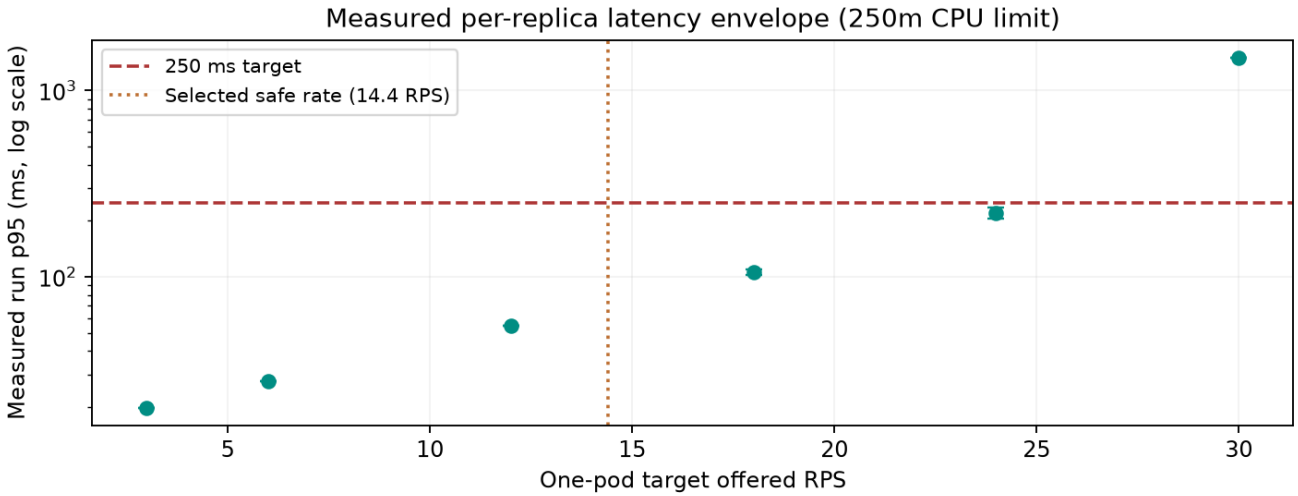


No future phase labels, scenario ID or scripted surge times are supplied to the predictor. The synthetic training family includes varied cycles, amplitudes and noise. It is controllable experimental data, not a production workload trace.

A separate recorded-replay candidate was subsequently trained on 32 measured-demand rows (seed 101), validated on 16 rows (202), and tested on 16 independent rows (404). Its held-out MAE was 4.96 RPS versus 11.06 for the original model and 10.65 for persistence. This small exploratory candidate was not substituted into the completed 12-run benchmark; its weights and provenance are saved separately.

Measured capacity and provisioning delay

We first measured one replica at increasing offered rates under the fixed 250m CPU limit. A rate passes only if every retained repeat meets global and nonempty-window p95 <=250 ms, successful responses >=99%, and the load generator remains valid. We select 80% of the highest repeatably passing rate as safe per-replica capacity.



Measurement	Observed outcome
18 RPS / one pod	Repeated passing runs; p95 approximately 103-109 ms and no failed requests.
24 RPS / one pod	Near the latency boundary; one repeat has violating windows despite global p95 below 250 ms. Not selected as repeatably safe.
30 RPS / one pod	CPU saturation; p95 about 1.5 s and approximately 2.63% failed requests.
Selected safe capacity	14.4 RPS per replica (0.8 x 18 RPS).
Capacity check	Two pods at 28.8 target RPS: p95 about 68 ms. Four pods at 57.6 target RPS: p95 about 110 ms. Both had 100% successful scored requests.

Across sampled scale-ups, window-end-to-Ready delay had median 8.9 s and p95 9.6 s. This includes decision/proxy latency and approximately one-second sampling granularity; it is not an exact boot-time measurement. The 30-second forecast horizon covers the observed provisioning delay.

Resource allocation and safety mechanism

Forecast requirement: ceil((predicted next-30s peak + validation buffer) / safe_rps).
Observed requirement: ceil(measured offered RPS / safe_rps).
The guarded controller selects the larger bounded requirement. With monotonically increasing capacity/cost, this chooses the smallest integer allocation covering that demand estimate; it is not a global optimization or reinforcement-learning algorithm.

Situation	Decision
Rising forecast	Request additional replicas immediately; distinguish desired, Ready and pending capacity. Do not treat pending pods as serving traffic.
Underprediction	Observed demand can override the forecast. If p95 exceeds 250 ms or error fraction exceeds 1%, request a bounded extra replica when no scale-out is pending.
Overprediction / falling load	Keep the maximum recommendation in the 30-second stabilization window; remove at most one replica per decision. Pending scale-outs block downscale.
Caps / budget	Clamp desired count to hardware maximum and any configured rate-budget cap. CAPACITY_LIMITED records an infeasible demand estimate; the controller cannot guarantee both the cap and the SLA.
Delayed/stale decision	The policy holds capacity when telemetry is marked stale. Unit tests exercise this guard. In the harness, live freshness is based on completed-window/decision lag; this is not a production monitoring-failure detector.
Failure handling	Actuation errors are recorded rather than treated as successful changes. Aborted experiments stop their generator, preserve available raw logs, and return the lab fleet to one replica.

Why stability matters

Readiness delay makes repeated blind scale-out requests wasteful. Immediate scale-down can also remove capacity just before a forecasted rise or while requests remain in flight. Common stabilization rules are applied to the dynamic baselines so the comparison does not falsely attribute all stability benefits to ML.

Unit and integration checks

Thirteen checks cover future-label/past-feature separation, repeatable seeds, saved-model equivalence, hard budget bounds, stale-metric hold, stabilization under clock jitter, pending-capacity handling, a real HTTP hash/CPU endpoint, and Kubernetes chunked JSON handling. These checks complement actual cluster runs; they do not replace performance measurements.

Matched experiment design

Dimension	Frozen comparison setting
Policies	Fixed maximum (4 pods), custom reactive scaling, guarded predictive scaling. The reactive baseline is not Kubernetes HPA.
Workloads	Periodic demand and an unseen flash-crowd addition. Arrivals are synthetic seeded Poisson schedules, replayed as real HTTP traffic.
Repeats	Two independent seeds (101, 202) per scenario/policy: 12 sequential runs. A saved seed randomizes policy run order.
Scoring	240-second run, first 60 seconds excluded as common model/controller warm-up; 180 scored seconds/run. Warm-up remains in raw logs.
Fairness	Same worker image, CPU task, CPU/memory limits, min/max replicas, ten-second control interval and stabilization settings. Each run resets to one Ready worker.
Generator checks	Raw planned versus actual sends, p99 scheduling lag, request status and bounded client concurrency. A generator-capacity drop invalidates a run.
Audit	4 scenario/seed schedule groups match byte-for-byte across policies; all 12 runs use identical model and source fingerprints.

Latency statistics use raw successful-request timings; failures/timeouts are reported separately so a low successful-only p95 cannot hide errors. SLA-window violations combine the latency target with the 99% success condition. We do not average window percentiles to calculate run p95.

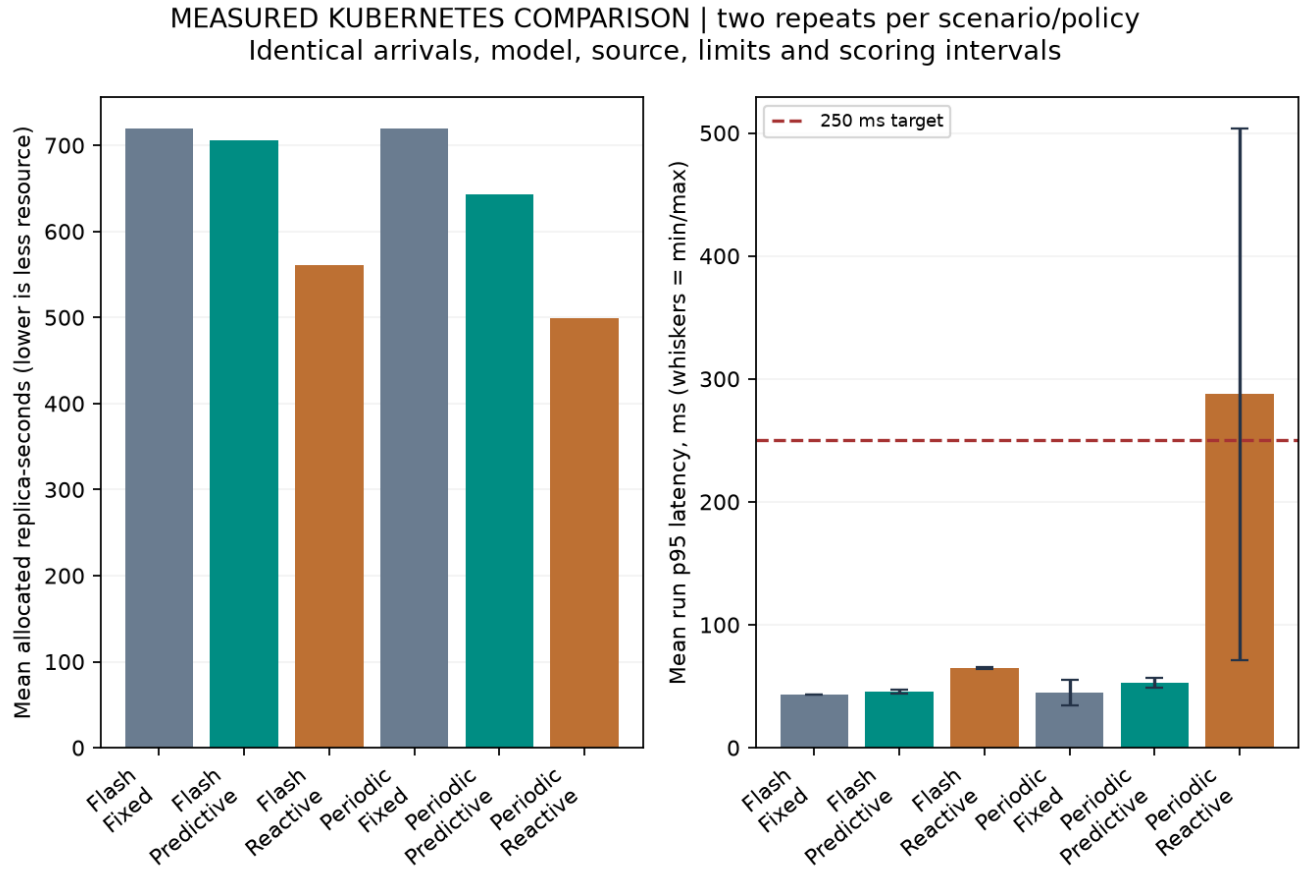
Allocated replica-seconds integrate actual pod allocation, including warming and terminating pods. Worker CPU is sampled from process CPU counters and normalized by the configured CPU limit. Constant gateway/node overhead is outside the worker-only resource comparison.

Cost = replica-hours x 0.02 illustrative currency units per replica-hour. Energy proxy = integral of 10 W per allocated replica plus 20 W per measured CPU core used, converted to Wh. Neither quantity represents actual AWS billing or physically measured energy; a shared host may consume similar wall power despite fewer pods.

Source SHA-256: 4f35ddcc9ef8c4ba77faa879974f604be4e3969dbbf60e1c26749703a998a3c4
Saved-model SHA-256: 5f927c47888aabb5c6a4b3a3997808c66c745d9ffc793fc6c25aaea50b42607a

Measured comparison results

Scenario / policy	Run p95 range (ms)	Replica-s mean	SLA windows mean
flash / fixed	42.8-43.2	720.0	0.0%
flash / guarded predictive	44.0-46.9	705.3	0.0%
flash / reactive	64.1-65.8	561.3	0.0%
periodic / fixed	34.6-55.3	720.0	0.0%
periodic / guarded predictive	49.0-56.6	643.1	0.0%
periodic / reactive	71.5-503.9	498.8	5.6%



Whiskers show the observed minimum/maximum over two repeats, not a statistical confidence interval. The plotted latency bar is the mean of run p95 values, explicitly separate from any pooled-request percentile. Two repeats are sufficient for a lab comparison, not a production generalization claim.

Utilization, estimated cost and energy

Scenario / policy	Mean CPU use / limit	Mean replica-s	Estimated cost	Energy proxy Wh
flash / fixed	27.2%	720.0	0.00400	2.272
flash / guarded predictive	28.1%	705.3	0.00392	2.234
flash / reactive	35.0%	561.3	0.00312	1.831
periodic / fixed	22.2%	720.0	0.00400	2.222
periodic / guarded predictive	25.4%	643.1	0.00357	2.013
periodic / reactive	31.9%	498.8	0.00277	1.607

CPU utilization is sampled worker process CPU time divided by allocated worker CPU-limit time (250m per pod). It excludes the gateway and Kubernetes system processes. More allocated headroom can lower utilization while protecting tail latency; maximum utilization is not itself the optimization objective.

Cost is in illustrative currency units, using 0.02 units/replica-hour for all policies. Only worker allocation is compared; constant platform overhead is excluded. The energy proxy assumes 10 W per allocated replica plus 20 W per measured CPU core used. These assumptions are not EC2 prices, physical host power, or actual energy savings.

Sensitivity to assumed idle power

Scenario / policy	0 W idle (Wh)	5 W idle (Wh)	20 W idle (Wh)
flash / fixed	0.272	1.272	4.272
flash / guarded predictive	0.275	1.254	4.193
flash / reactive	0.272	1.052	3.390
periodic / fixed	0.222	1.222	4.222
periodic / guarded predictive	0.227	1.120	3.799
periodic / reactive	0.221	0.914	2.992

With zero assumed idle power, the proxy mainly reflects the similar CPU work completed by each policy, and differences shrink. With a higher idle term, fewer allocated replicas appear more favorable. This is sensitivity analysis of a model, not a measurement of the laptop's wall power.

Trade-offs, prediction error and limitations

Periodic workload

Guarded prediction used 10.7% fewer worker replica-seconds than fixed maximum capacity, but 28.9% more than the reactive baseline. Its run p95 range was 49.0-56.6 ms; reactive was 71.5-503.9 ms. These measurements expose the cost/performance trade-off rather than establish a universal winner.

The operating predictor's mean MAE was 9.42 RPS, versus persistence 9.71 RPS. Synthetic holdout accuracy alone therefore does not establish superior forecasting during real replay.

Flash workload

Guarded prediction used 2.0% fewer worker replica-seconds than fixed maximum capacity, but 25.7% more than the reactive baseline. Its run p95 range was 44.0-46.9 ms; reactive was 64.1-65.8 ms. These measurements expose the cost/performance trade-off rather than establish a universal winner.

The operating predictor's mean MAE was 13.07 RPS, versus persistence 9.07 RPS. Synthetic holdout accuracy alone therefore does not establish superior forecasting during real replay.

When the prediction is wrong

An unseen burst cannot reliably be predicted from a history that contains no warning. The observed-demand/latency path can request more capacity, but pod readiness delay still permits temporary harm. Overprediction consumes extra replica-time until stabilization allows downscale. A hard cap can make the latency goal infeasible; the correct response is to expose that conflict, not claim an unconditional SLA guarantee.

Energy and cost sensitivity

The energy proxy weights allocated idle capacity and actual CPU separately. Changing assumed idle watts changes the apparent benefit of fewer replicas; removing the idle term greatly reduces that benefit because the same CPU work still executes. No physical energy reduction or cloud-bill saving is inferred from this local experiment.

Limits and next improvements

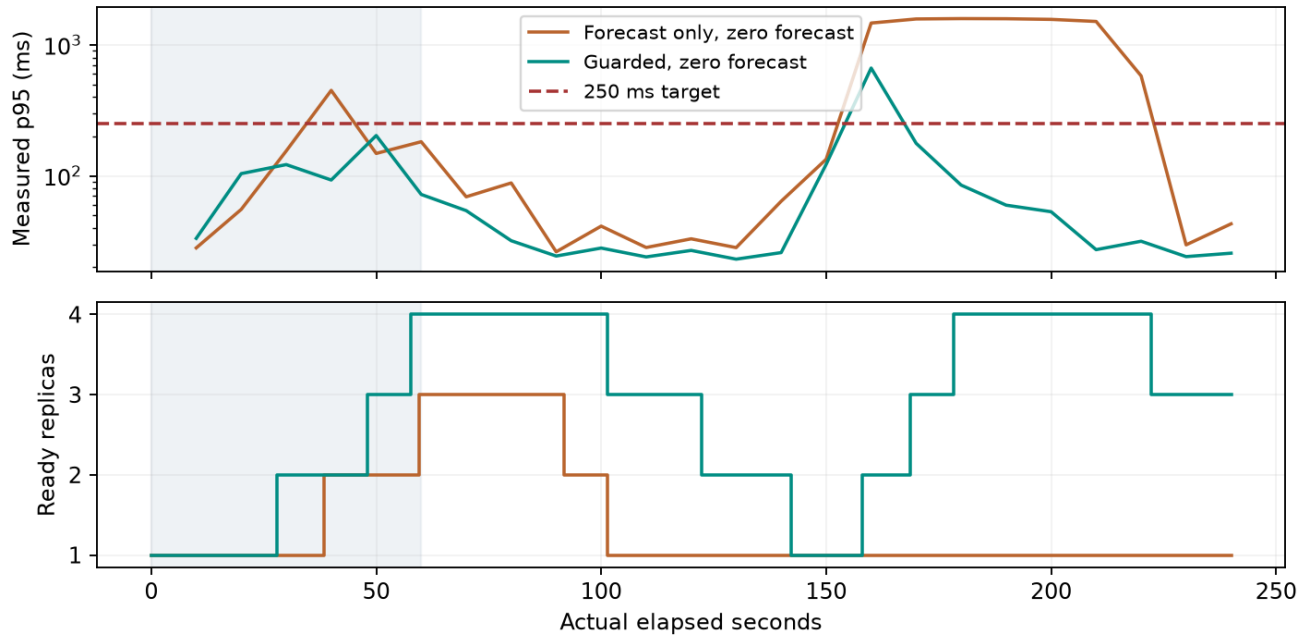
The workload is synthetic, training is offline, and all replicas share one laptop. Only two test seeds are used. Host contention, Service routing and readiness sampling affect variability. Future work would validate on independent production traces, recalibrate after deployment changes, improve uncertainty estimation, and test a real cloud billing/energy environment. These are future extensions, not implemented claims.

Deliberate prediction failure and a hard budget

Injected condition / policy	Run p95 ms	Failed requests	SLA windows
zero-forecast stress / plain predictive	1547.7	13.35%	38.9%
zero-forecast stress / guarded predictive	215.1	0.00%	5.6%
one-replica budget stress / guarded predictive	1563.2	17.73%	66.7%

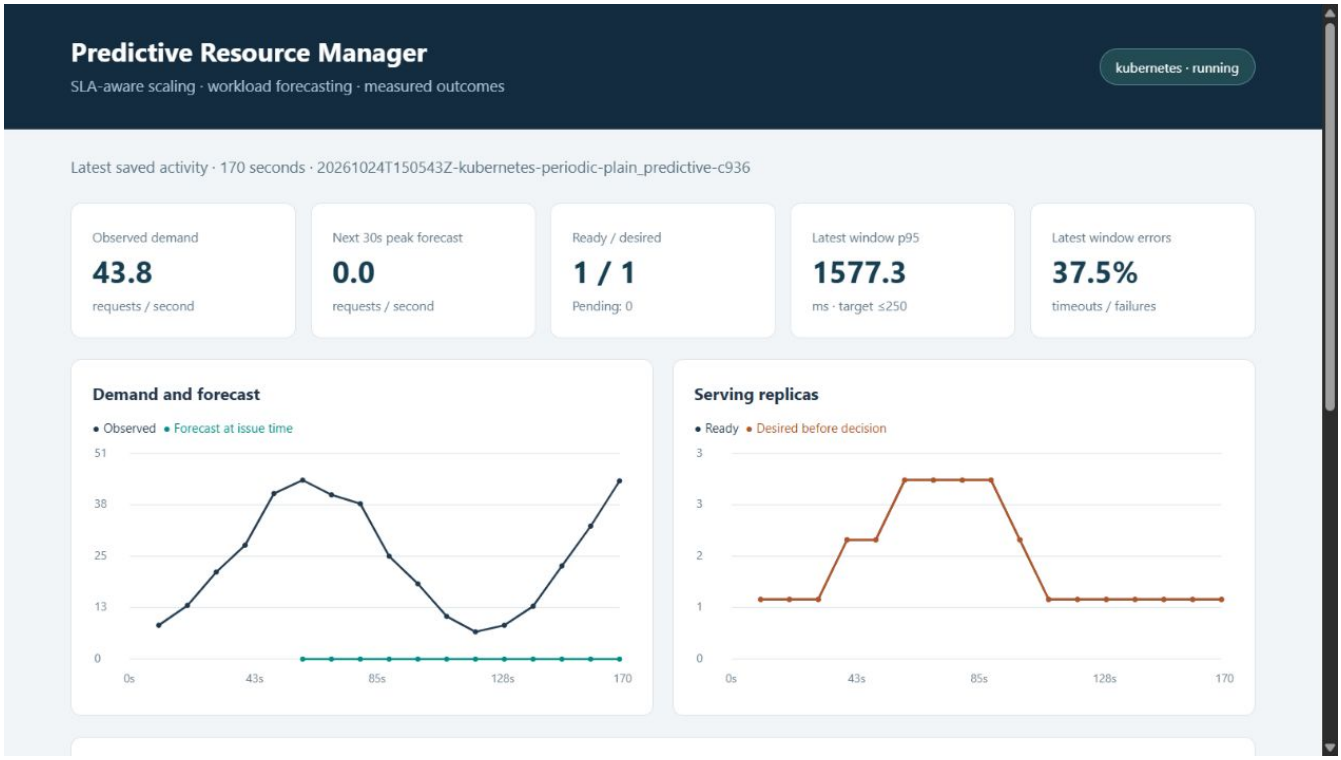
The zero-forecast pair deliberately multiplies the predictor output by zero while replaying the same periodic arrival schedule (seed 404). This is a controlled model-failure injection, not a claim about naturally observed model accuracy. We compare the forecast-only ablation against the guarded policy; both retain common readiness, replica limits and stabilization. One run per policy is used for this mechanism demonstration.

In this paired injection, forecast-only control failed 13.35% of requests with p95 1548 ms. Guarded control failed 0.00% with p95 215 ms. Its violating-window rate was still 5.6%, demonstrating that rescue cannot remove readiness delay.

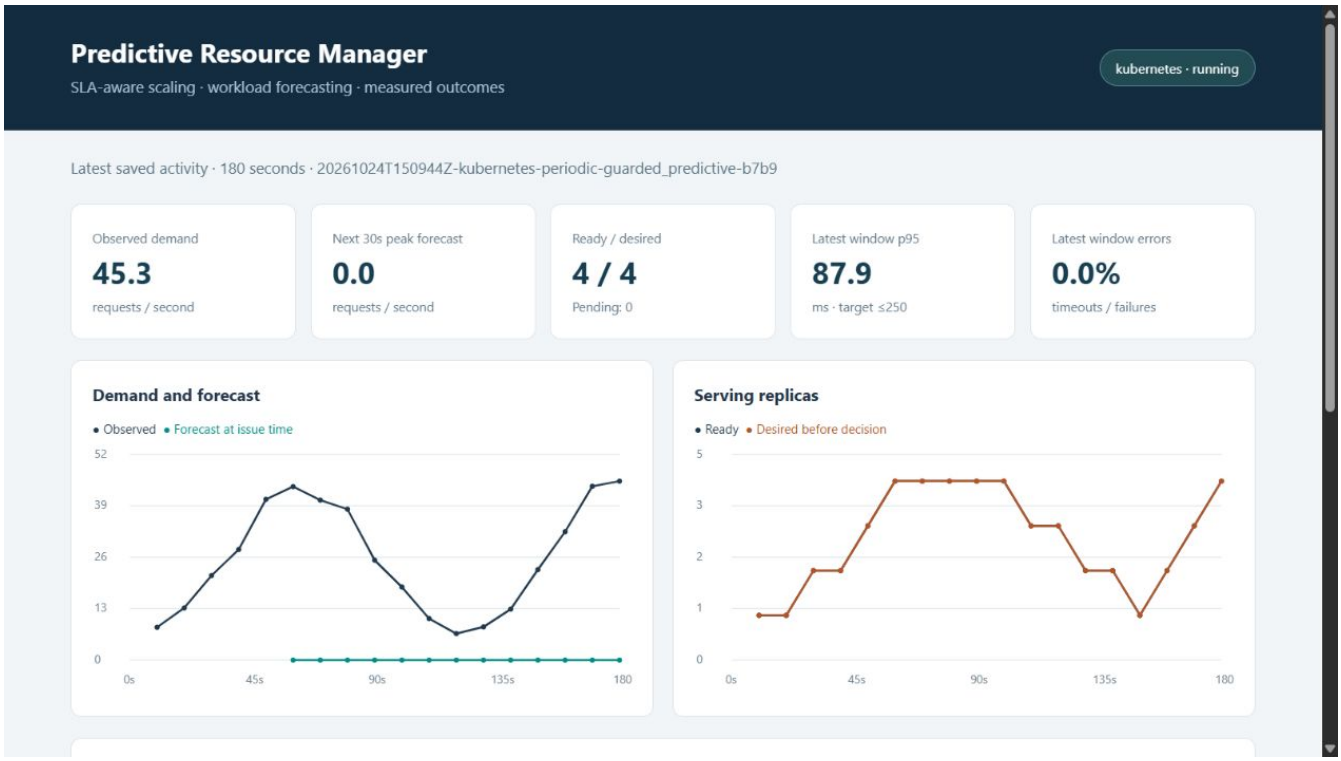


The separate budget stress test uses a different seed (505), a 120-second run and a rate budget of 0.02 units/hour at 0.02 units/replica-hour, limiting desired capacity to one replica. Its outcome demonstrates the budget/SLA conflict; it is not a paired performance comparison against the zero-forecast runs. A resource ceiling must be exposed when it prevents sufficient provisioning.

Execution screenshots: failed forecast and rescue

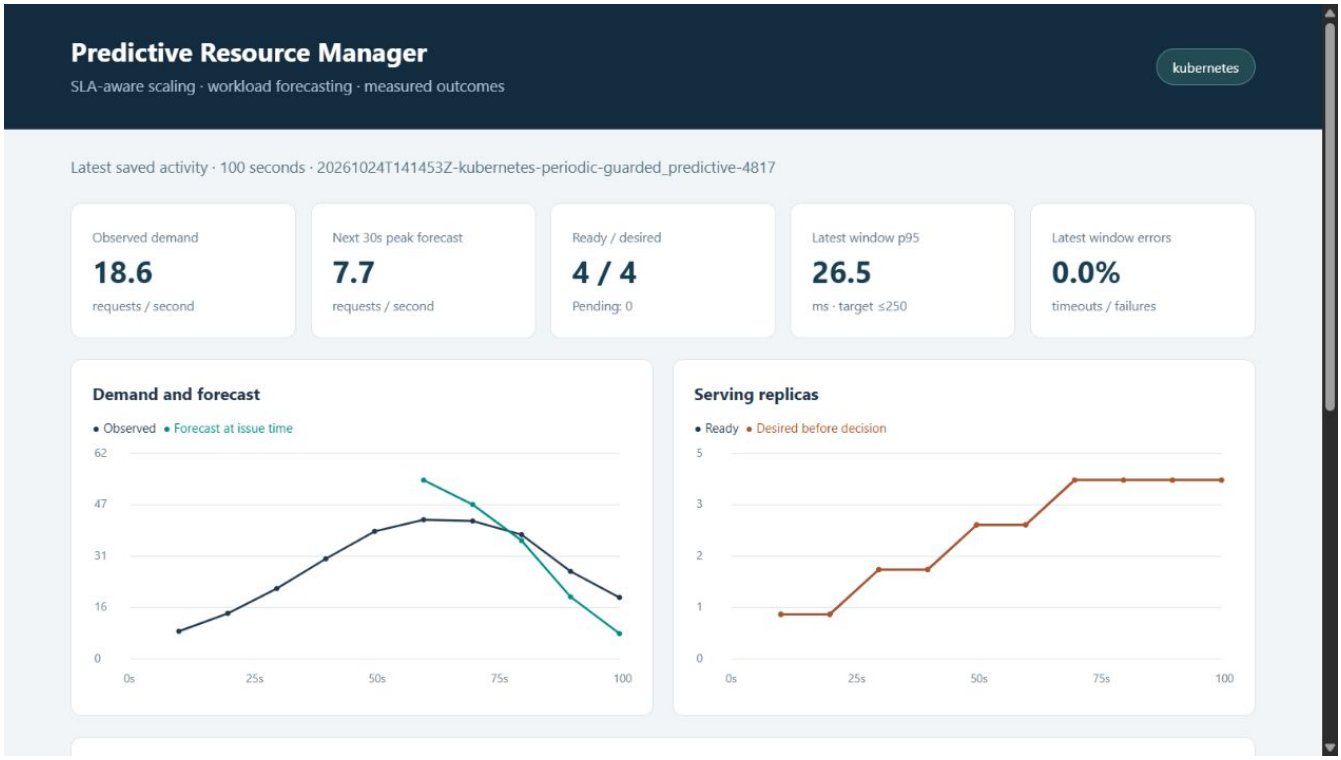


Forecast-only ablation during the real zero-output injection: the model output is zero while actual incoming requests continue. The resource controller retains its minimum allocation. Measured failure and latency totals are reported on the preceding page.



Guarded policy under the same injected condition: observed demand and latency can request capacity independently of the zero forecast. This screenshot records a real rescue transition; temporary latency harm remains visible in the full raw timeline.

Execution evidence and reproduction



Actual application viewer during a Kubernetes run: real desired/Ready replicas, measured offered RPS and latency, and the model forecast at issue time. This is execution evidence, not a simulated capacity plot. The raw deployment listing is preserved in output/evidence/kubernetes_state.txt.

Reproduction

1. Install requirements; run SetupLab.ps1.
2. Calibrate the current machine and train the saved model at its safe rate.
3. Run scaler.matrix for matched experiments, then tools/evaluate_matrix.py.
4. Inspect per-run requests.csv, decisions.csv, resources.csv, model.json and summary.json.
5. Run scaler.viewer for the read-only live/result display. README contains exact commands.

References

Course Lab-CA assignment and submission rubric, supplied October 2026.
Kubernetes: Horizontal Pod Autoscaling (background; our reactive baseline is custom).
Minikube: Docker driver.
scikit-learn: Lagged features for time-series forecasting (causal evaluation guidance; our Ridge implementation uses NumPy).